COST-AWARE HEART-FAILURE MORTALITY PREDICTION WITH A SMALL NEURAL NETWORK

William Shoaf
Lane Department of Computer Science and Electrical Engineering
West Virginia University
CPE 520: Applications of Neural Networks, Midterm Project
Professor Anthony Sicilia

ABSTRACT

A ReLU network was trained to estimate the probability of death during follow-up for 299 heart-failure patients from 11 routine clinical measurements. Four controlled experiments were conducted to study modeling parameter choices that follow from detected problems in the data and the clinical deployment setting. (Exp 1) Oversampling deaths raised recall from 0.44 to 0.69. (Exp 3) L2 and dropout each helped reduce overfitting fro the small sample size (n=299). (Exp 4) Standardizing inputs was necessary for training to be better than always guessing the majority class. (Exp 2) Comparison of decision threshold choices showed that with a cost matrix that assigned a missed death to cost $20\times$ a false alarm, the default $\tau = 0.5$ cost 3.71 per patient versus 0.72 for the Bayes-optimal threshold $\tau^* = 1/21$. The Bayes-optimal threshold also outperformed a threshold tuned on the small validation split. However, at that cost ratio the model did not beat flagging every patient. At 5:1 ($\tau^* = 1/6 \approx 0.17$), both cost-aware thresholds cut cost by about 35% relative to $\tau = 0.5$ (0.65–0.67 vs. 1.00), and the tuned and theoretical thresholds performed equivalently (p=.54). Because false alarms are relatively more expensive at this ratio, the model now roughly ties flagging everyone (0.65 vs. 0.68) while raising 44% fewer alerts, showing that the model's practical value is dependent upon the assigned cost matrix.

1 PROBLEM DESCRIPTION

MOTIVATION

Heart-failure clinics can sometimes miss when clinical data indicates a need for further care. In this study a decision-support tool is studied that estimates each patient's probability of death during follow-up, using clinical measurements recorded at admission. The probability of death during followup is given by $\hat{p} = P(\text{death} \mid \boldsymbol{x})$, where x is the given clinical records. The tool then flags the patient for closer monitoring when $\hat{p} \geq \tau$, a set decision threshold. The UCI Heart Failure Clinical Records dataset, retrieved from UCI Machine Learning Repository (UCI, 2020) and originally collected by Ahmad et al. (Ahmad et al., 2017), is used and consists of data from 299 patients admitted in 2015 with a binary outcome `DEATH_EVENT` (96 deaths out of 299). The inputs are 6 continuous measurements (age, ejection fraction, serum creatinine, serum sodium, platelets, creatinine phosphokinase) and 5 binary indicators (anaemia, diabetes, high blood pressure, sex, smoking), for a total of 11 features. These are established clinical risk markers, and Chicco & Jurman (2020) showed that ejection fraction and serum creatinine alone predict survival on this specific dataset; their best model, a random forest, reached ROC AUC 0.800. The data can therefore support the prediction, and utilizing models to flag potentially missed clinical observations could save lives.

DATA PROBLEMS

Label leakage. Although we only use 11 features, the dataset also includes `time`. This is the follow-up length (4-285 days), which in the case of a death event, ends at death. Therefore, short

follow-up shows the model to predict death, and `time` is unknown when the prediction would be made, so it is excluded.

Class imbalance. There are 203 survivors versus 96 deaths in the 299 patient dataset. If the model predicts "survived" for everyone it scores 67.9% accuracy while not giving any useful clinical information. Therefore, accuracy alone is an uninformative metric and an unweighted loss is dominated by survivors (Exp 1).

Small sample. Each cross-validation round trains on only ≈191 patients, so overfitting is likely to occur(Exp 3). A 60-patient test fold contains only ≈19 deaths, so each missed death moves recall by about 5 points. Therefore variability from fold to fold is measured.

Mismatched feature scales and outliers. Platelets have mean $263{,}358$ with a standard deviation of $97{,}804$, while serum creatinine has mean 1.39 and a standard deviation of 1.03. This difference in scale is quite large and could slow down the model's convergence. Therefore, we test how feature scaling affects the model in experiment 4.

Hidden imputation. The file reports no missing values, but 25 platelet values equal the column mean ($263{,}358.03$) and are fractional, which no lab measurement of platelets can produce. The original authors may have imputed missing platelet counts with the mean. No other feature seems to have imputed values, and and platelets are a weak predictor (Chicco & Jurman, 2020) so no modifications are made around this.

Deployment Problems

Asymmetric error costs. Failing to flag a patient who dies (false negative) is far worse than a false alarm (false positive) leading to an earlier followup or more clinical attention. The cost of these decisions is described with costs $C_{\text{FN}} = 20$, $C_{\text{FP}} = 1$, and correct decisions costing 0. The decision threshold is described by its expected cost per patient,

$$\text{Cost}(\tau) = \frac{C_{\text{FN}}\,\text{FN}(\tau) + C_{\text{FP}}\,\text{FP}(\tau)}{N}. \tag{1}$$

This equation is derived from Elkan's optimal decisions for cost matrix properties (Elkan, 2001) and is described in further detail in the appendix. Because the ratio 20:1 is an assumption, we also test 5:1 and 10:1 (Experiment 2).

Limited staff and alert fatigue. Every flag from the model consumes clinician time, so a rule that flags nearly everyone may be perceived as a "boy who cried wolf" scenario to the clinicians. Testing different cost ratios allows for seeing the impact on how often the model flags depending on different costs.

Single site. All records come from one hospital in one year, collected by one group of people. Therefore, results may not transfer to other populations. This is a clear limitation on the model and shows a need to test the model on other, diverse, datasets before implementation.

2 Experiment 1

Research Question

Does randomly oversampling deaths in the training data improve recall on deaths, compared to training on the original 32% death rate, and at what cost to precision and specificity?

Experimental Design

Shared protocol (all experiments). The base model is a ReLU network with hidden widths $(32, 16)$: $\boldsymbol{h}_1 = \text{ReLU}(\boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1)$, $\boldsymbol{h}_2 = \text{ReLU}(\boldsymbol{W}_2\boldsymbol{h}_1 + \boldsymbol{b}_2)$, $s = \boldsymbol{w}_3^\top \boldsymbol{h}_2 + b_3$, $\hat{p} = \sigma(s)$, trained with binary cross-entropy (BCE) and Adam (Kingma & Ba, 2015) (learning rate 10^{-3}, batch size 32) in PyTorch (Paszke et al., 2019). Stratified 5-fold cross-validation was used. Within each round, 20% of the training folds is held out as a validation split for early stopping (patience 20, at most 300 epochs). Continuous features are standardized with statistics fit on the training rows only, and binary features remained 0/1. All conditions share identical splits and initial weights, so we report mean ± s.d. over

the 5 test folds and the paired fold-by-fold difference with a paired t-test. With $n = 5$ the test has low power, so we also report in how many folds the difference has the same sign.

This experiment. Condition A trains on the original data. Condition B duplicates training-split deaths at random with replacement until the classes are equal, which is a supported method (He & Garcia, 2009). Validation and test folds are never resampled. Metrics are computed at $\tau = 0.5$. Recall shows how many actual positives found. Precision shows how many predicted positive were actually positive. F1 is harmonic mean of precision and recall. Specificity shows how many actual negative instances were found. ROC AUC measures how well the model ranks predictions. PR AUC shows performance of positive class.

Results and Analysis

Table 1: Experiment 1 (test folds, $\tau = 0.5$, mean $\pm$ s.d. over 5 folds). Δ: paired difference B$-$A, with the number of folds in which B is higher; *paired t-test $p < 0.05$.

	Recall	Precision	F1	Specificity	ROC AUC	PR AUC
A: original	0.44 ± 0.09	0.64 ± 0.17	0.51 ± 0.08	0.86 ± 0.09	0.75 ± 0.08	0.62 ± 0.15
B: oversampled	0.69 ± 0.14	0.61 ± 0.16	0.64 ± 0.12	0.78 ± 0.13	0.77 ± 0.07	0.63 ± 0.16
Δ (B$-$A)	$+0.25^*$ (5/5)	-0.02 (1/5)	$+0.13$ (5/5)	-0.08 (0/5)	$+0.02$ (4/5)	$+0.01$ (3/5)

Oversampling raised recall in every fold, from 0.44 to 0.69 (Table 1). Summed over all test folds, the model caught 66 of the 96 deaths instead of 42, at the price of 45 false alarms instead of 28. Precision was essentially unchanged and specificity fell by 8 points, so the answer to the research question is yes. Oversampling improves recall on deaths at a cost to specificity.

Ranking quality, however, barely changed (ROC AUC $+0.02$, PR AUC $+0.01$). Oversampling multiplies the weight of deaths in the BCE loss, which raises every predicted probability rather than teaching the network to separate patients better.

3 Experiment 2

Research Question

With a missed death(FN) costing $20\times$ a false alarm(FP), which decision threshold minimizes the expected cost of Eq. 1, how much does it save relative to the default $\tau = 0.5$, and can the optimal threshold be derived from the costs alone?

Experimental Design

Theoretical threshold. For a patient with predicted risk $\hat{p}$, flagging has expected cost $(1 - \hat{p})\, C_{\text{FP}}$ and not flagging has expected cost $\hat{p}\, C_{\text{FN}}$. Flagging is cheaper exactly when

$$\hat{p} > \tau^* = \frac{C_{\text{FP}}}{C_{\text{FP}} + C_{\text{FN}}} = \tfrac{1}{21} \approx 0.048, \tag{2}$$

the minimum-expected-cost (Bayes) decision rule (Elkan, 2001). Equal costs for a false negative and false positive yield $\tau^* = 0.5$ by the expected cost per patient equation. Eq. 2 is optimal only if $\hat{p}$ is calibrated, which neural networks often are not (Guo et al., 2017).

Procedure. Reuse the base model of Experiment 1 without retraining. In each fold, the tuned threshold $\hat{\tau}^*$ is the tested value in $\{0.01, 0.02, \ldots, 0.95\}$ that minimizes Eq. 1 on the validation split. On the test fold we compare four decision thresholds: the default threshold, $\tau = 0.5$, the tuned threshold, $\hat{\tau}^*$, the theoretical optimal threshold, τ^*, and "flag everyone" ($\tau = 0$). We repeat for cost ratios $5{:}1$ and $10{:}1$ and check calibration with a reliability diagram and the Brier score, which measures how accurate a model's predicted probabilities are (Brier, 1950).

Table 2: Experiment 2: test-fold cost per patient (Eq. 1, mean $\pm$ s.d.) for each policy and cost ratio. False positives (FP) and false negatives (FN) per fold are shown for 20:1.

	Cost per patient			At 20:1		
Policy	5:1	10:1	20:1	Recall	FP	FN
Default $\tau = 0.5$	1.00 ± 0.15	1.90 ± 0.31	3.71 ± 0.62	0.44	5.6	10.8
Tuned $\hat{\tau}^*$	0.67 ± 0.11	0.86 ± 0.21	1.29 ± 0.55	0.86	25.4	2.6
Theoretical τ^*	0.65 ± 0.11	0.71 ± 0.26	0.72 ± 0.14	0.98	35.2	0.4
Flag everyone	0.68 ± 0.01	0.68 ± 0.01	0.68 ± 0.01	1.00	40.6	0.0

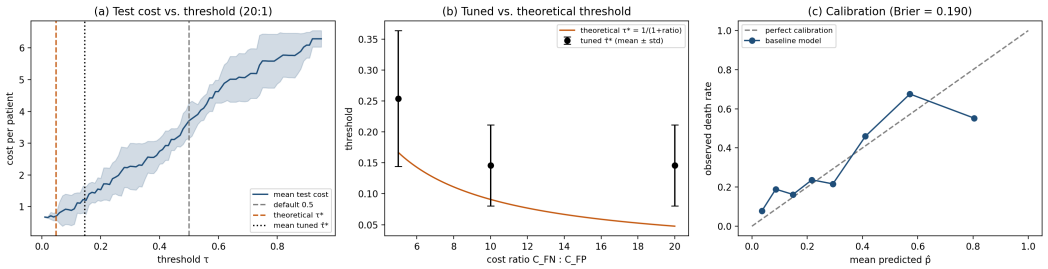


Figure 1: Experiment 2. (a) Mean test cost versus threshold at 20:1 (± 1 s.d. band); cost rises almost linearly from $\tau = 0$. (b) Tuned thresholds versus the theoretical curve $\tau^* = 1/(1 + \text{ratio})$. (c) Reliability diagram of pooled out-of-fold test predictions.

Results and Analysis

The default threshold is a poor decision threshold. At 20:1, every alternative cut cost by 65–82% relative to $\tau = 0.5$ (tuned: -2.41, p=.005; theoretical: -2.98, p<.001; lower in 5/5 folds for both). The default misses more than half the deaths (10.8 of $\approx$19 per fold), each costing 20 units.

The cost-derived theoretical optimum threshold beat the tuned one. At 20:1, τ^* cost 0.57 less than $\hat{\tau}^*$ per patient (lower in 4/5 folds, p=.047). Each validation split holds only $\approx$15 deaths, and at high cost ratios the cost-minimizing grid value is simply the highest threshold that still catches every validation death. It is therefore set by a single patient, varies widely across folds (0.07–0.24), and was identical at 10:1 and 20:1 (Fig. 1b). On the test folds it then missed 2.6 deaths per fold. Eq. 2 needs no data at all and so cannot overfit the small validation split, whereas the tuned threshold may suffer from overfitting. Its success also supports the calibration of the low-probability region, meaning that among patients assigned a predicted risk of about 5%, that a death event happened about 5% of the time. The reliability diagram (Fig. 1c) agrees in that it follows the diagonal up to $\hat{p} \approx 0.6$ and is overconfident only above that, and the Brier score (0.190) improves on the 0.218 of a constant base-rate prediction.

At 20:1 the model does not beat flagging everyone. The theoretical threshold's cost (0.72) is statistically very similar to flagging all patients (0.68; p=.53), and the test-cost curve (Fig. 1a) is lowest at $\tau \to 0$. With ROC AUC ≈ 0.75, the network cannot rule out any patient confidently enough to justify a 1-in-21 risk of a missed death. At 5:1, τ^* roughly ties flag-everyone (0.65 vs. 0.68). The value of the model therefore depends on how expensive alerts really are: it becomes useful only when limits on staff capacity makes false alarms costly.

4 Experiment 3

Research Question

With $\approx$191 training patients, do L2 regularization or dropout reduce the gap between training and validation loss and improve held-out loss as compared to the base model with no regularization?

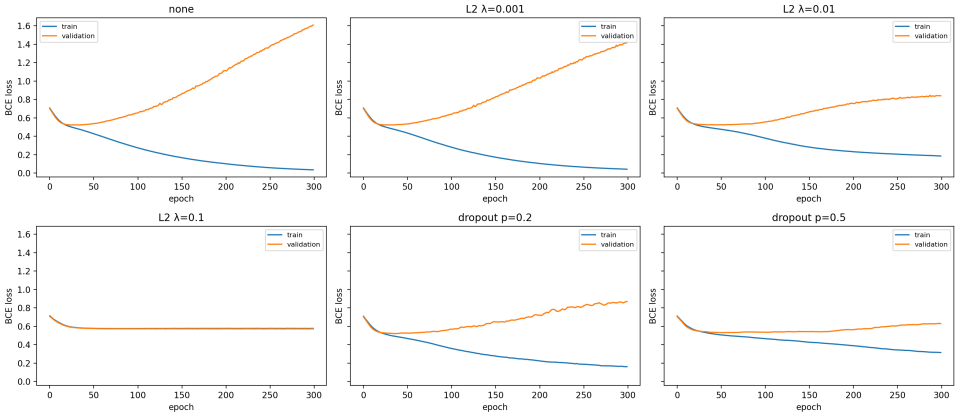


Figure 2: Experiment 3: training and validation BCE over 300 epochs (fold 1). Without regularization, validation loss reaches its minimum near epoch 25 and then rises while training loss approaches zero.

Experimental Design

Early stopping is itself a form of regularization and could mask overfitting, so all runs in this experiment train for the full 300 epochs, and losses are recorded at the final epoch. L2 is added directly to the objective, $J(\theta) = \text{BCE}(\theta) + \frac{\lambda}{2}\sum_\ell \|\boldsymbol{W}_\ell\|_F^2$ (weights only), with $\lambda \in \{10^{-3}, 10^{-2}, 10^{-1}\}$. Dropout (Srivastava et al., 2014) with $p \in \{0.2, 0.5\}$ follows each hidden ReLU. This study reports training and validation BCE (data loss only, dropout off), their gap, and test BCE and ROC AUC.

Results and Analysis

Table 3: Experiment 3 (300 epochs, no early stopping; mean $\pm$ s.d. over 5 folds). Every regularized condition reduced the gap and test BCE in 5/5 folds ($p < 0.04$ for all).

Condition	Train BCE	Val BCE	Gap	Test BCE	ROC AUC	Recall@0.5
None	0.06	1.17 ± 0.42	1.12 ± 0.43	1.34 ± 0.42	0.68 ± 0.07	0.47
L2 $\lambda = 10^{-3}$	0.07	1.01 ± 0.38	0.93 ± 0.39	1.13 ± 0.36	0.71 ± 0.06	0.49
L2 $\lambda = 10^{-2}$	0.24	0.59 ± 0.17	0.35 ± 0.21	0.69 ± 0.16	0.72 ± 0.07	0.44
L2 $\lambda = 10^{-1}$	0.59	0.58 ± 0.03	0.00 ± 0.01	0.59 ± 0.02	0.78 ± 0.08	0.04
Dropout $p = 0.2$	0.18	0.62 ± 0.18	0.43 ± 0.20	0.79 ± 0.30	0.73 ± 0.08	0.49
Dropout $p = 0.5$	0.32	0.51 ± 0.08	0.19 ± 0.10	0.62 ± 0.17	0.75 ± 0.08	0.48

The base model with no early stopping shows signs of extreme overfitting: training BCE falls to 0.06 while validation BCE climbs to 1.17 (Fig. 2). Both L2 and dropout reduce overfitting and their effect grows with strength. Every regularized condition reduced the gap and the test BCE in 5/5 folds (Table 3), and dropout p=0.5 cut the gap from 1.12 to 0.19 while raising AUC from 0.68 to 0.75. This matches the view of dropout as training-time pruning that lowers the effective parameter count. Also, the early-stopped base model of Experiment 1 (test BCE 0.57, AUC 0.75) matches the best results here, so early stopping alone can prevent most of the overfitting.

5 Experiment 4

Research Question

Does standardizing the six continuous inputs reduce the number of epochs needed to converge and improve validation loss compared to feeding raw values?

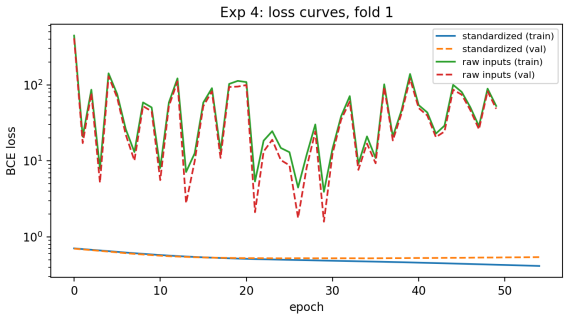


Figure 3: Experiment 4: BCE (log scale) for fold 1. Raw inputs produce losses that oscillate between $\approx$2 and several hundred; standardized inputs train smoothly.

Experimental Design

We compare the base model (standardized continuous inputs, scaler fit on training rows) with an identical model on raw inputs, using the same splits, seeds, optimizer and early-stopping rule. Convergence is measured by the epoch of minimum validation loss. We also report test BCE, ROC AUC and accuracy. The design is motivated by the first-layer gradient, $\partial s_j/\partial W_{jk} = x_k$, which is proportional to each input: with raw platelet counts near 2.6×10^5 and creatinine near 1, one learning rate cannot suit both weights (LeCun et al., 1998).

Results and Analysis

Table 4: Experiment 4 (mean $\pm$ s.d. over 5 folds). Δ: paired difference raw$-$standardized, with the number of folds in which raw is higher; *paired t-test $p < 0.05$.

Inputs	Best epoch	Val BCE	Test BCE	ROC AUC	Accuracy
Standardized	60 ± 31	0.46 ± 0.05	0.57 ± 0.09	0.75 ± 0.08	0.73 ± 0.06
Raw	27 ± 14	3.20 ± 2.54	2.94 ± 1.09	0.49 ± 0.10	0.53 ± 0.08
Δ	-33 (0/5)	$+2.74$ (5/5)	$+2.37^*$ (5/5)	-0.26^* (0/5)	-0.19^* (0/5)

Standardization is necessary on this dataset for the network to learn. With raw inputs, the loss oscillated from epoch to epoch (Fig. 3), validation BCE was $7\times$ higher, ROC AUC fell to 0.49 (chance level), and accuracy (0.53) was below the 0.68 of always predicting "survived" (Table 4). The raw-input runs did reach their best validation epoch sooner (27 vs. 60), but at a far worse loss. Early stopping simply halted an unstable run, so this is not faster convergence. The answer to the research question is therefore yes for validation loss and not meaningful for epochs. Standardization does not necessarily speed up training here, but without it no meaningful training occurs. Despite Adam's adaptive steps, a platelet input of $\sim 10^5$ multiplied by an initial weight of order 0.1 gives pre-activations in the tens of thousands, affecting the output and producing the large, erratic gradients seen in Fig. 3.

Summary and Limitations

Standardization (Exp. 4) and regularization or early stopping (Exp. 3) are necessary for a usable model on this dataset. Oversampling (Exp. 1) catches more deaths by shifting predictions rather than improving the ranking. The default decision threshold (Exp. 2), $\tau = 0.5$ is far from optimal under asymmetric costs, and the cost-derived τ^* outperforms a threshold tuned on a small validation split. At a 20:1 cost the model's ranking (AUC ≈ 0.75) is not strong enough to beat flagging every patient, so other cost ratios should be chose by the clinicians. Limitations: one shuffle of 5 folds gives low-powered tests; 25 platelet values are imputed; the cost ratio is assumed; and all data come from a single hospital.

REFERENCES

Tanvir Ahmad, Assia Munir, Sajjad Haider Bhatti, Muhammad Aftab, and Muhammad Ali Raza. Survival analysis of heart failure patients: A case study. *PLoS ONE*, 12(7):e0181001, 2017.

Anthropic. Claude Opus 5.5. Large language model, `https://claude.ai`, 2026. Accessed October 2026.

Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.

Davide Chicco and Giuseppe Jurman. Machine learning can predict survival of patients with heart failure from serum creatinine and ejection fraction alone. *BMC Medical Informatics and Decision Making*, 20(16), 2020.

Charles Elkan. The foundations of cost-sensitive learning. In *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 973–978, 2001.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.

Haibo He and Edwardo A. Garcia. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

Yann LeCun, Léon Bottou, Genevieve B. Orr, and Klaus-Robert Müller. Efficient backprop. In *Neural Networks: Tricks of the Trade*, pp. 9–50. Springer, 1998.

Adam Paszke et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15:1929–1958, 2014.

UCI. Heart Failure Clinical Records. UCI Machine Learning Repository, 2020. DOI: https://doi.org/10.24432/C5Z89R.

A Derivation of Eq. 1 - Expected Cost per Patient

Cost-sensitive decision rule. Following Elkan (2001), let $C(i, j)$ be the cost of predicting class i when the true class is j, with $i, j \in \{0, 1\}$ and $1 =$ death. For a patient with features $\boldsymbol{x}$, the expected cost (conditional risk) of predicting class i is

$$L(\boldsymbol{x}, i) = \sum_{j \in \{0,1\}} P(j \mid \boldsymbol{x})\, C(i, j), \tag{3}$$

and the optimal decision is the class with the smaller $L(\boldsymbol{x}, i)$. Writing $p = P(1 \mid \boldsymbol{x})$, predicting 1 (flagging) is optimal when $L(\boldsymbol{x}, 1) \leq L(\boldsymbol{x}, 0)$, i.e.

$$(1 - p)\, C(1, 0) + p\, C(1, 1) \;\leq\; (1 - p)\, C(0, 0) + p\, C(0, 1), \tag{4}$$

which gives Elkan's threshold

$$p^* = \frac{C(1, 0) - C(0, 0)}{C(1, 0) - C(0, 0) + C(0, 1) - C(1, 1)}. \tag{5}$$

Specializing Expected Cost per Patient. Correct decisions cost 0, $C(0, 0) = C(1, 1) = 0$, a missed death costs $C(0, 1) = C_{\text{FN}} = 20$, and a false alarm costs $C(1, 0) = C_{\text{FP}} = 1$. Eq. 3 then reduces to the per-patient expected costs

$$L(\boldsymbol{x}, 0) = p\, C_{\text{FN}} \qquad \text{(not flagged: cost } C_{\text{FN}} \text{ if the patient dies)}, \tag{6}$$

$$L(\boldsymbol{x}, 1) = (1 - p)\, C_{\text{FP}} \qquad \text{(flagged: cost } C_{\text{FP}} \text{ if the patient survives)}, \tag{7}$$

and Eq. 5 reduces to

$$\tau^* = \frac{C_{\text{FP}}}{C_{\text{FP}} + C_{\text{FN}}} = \frac{1}{21} \approx 0.048. \tag{8}$$

From expected cost to measured cost per patient. Eq. 3 is an expectation for a single patient and requires the unknown $P(j \mid \boldsymbol{x})$. To evaluate a threshold rule $\hat{y}_i = \mathbf{1}\{\hat{p}_i \geq \tau\}$ on N patients with known outcomes y_i, we replace the expectation by the realized cost $C(\hat{y}_i, y_i)$ of each decision and average over patients (the empirical risk):

$$\text{Cost}(\tau) = \frac{1}{N} \sum_{i=1}^{N} C(\hat{y}_i, y_i) \tag{9}$$

$$= \frac{1}{N} \Big[C(0, 1) \sum_i \mathbf{1}\{\hat{y}_i = 0, y_i = 1\} + C(1, 0) \sum_i \mathbf{1}\{\hat{y}_i = 1, y_i = 0\} \Big] \tag{10}$$

$$= \frac{C_{\text{FN}}\, \text{FN}(\tau) + C_{\text{FP}}\, \text{FP}(\tau)}{N}, \tag{11}$$

where the second line uses $C(0, 0) = C(1, 1) = 0$ and the third counts missed deaths and false alarms. This is the expected-cost-per-patient metric used to compare thresholds.

Use of AI Assistance

This project used Claude, an AI assistant (Anthropic, 2026), to facilitate a more efficient workflow for the author. Claude's role helped refine the design of the four experiments, find reference material, generate PyTorch code, format LaTeX, and helped interpret results.